

CS 61A — Topics, Priorities, and Workload Plan

Topics, Priorities, and Workload Plan

1. Course information

- **Course:** Computer Science 61A — *The Structure and Interpretation of Computer Programs*
- **Languages:** Python (primary), Scheme, SQL
- **Course site:** <https://cs61a.org/> (Spring 2026)
- **Final exam: Tue May 12, 2026** — exact time/location: verify at <https://cs61a.org/articles/about-61a/> before Day 1.
 - Format: in-person, written. Expect ~3 hours.
 - Cheat sheet: course norm is 2 double-sided pages of handwritten notes — confirm for SP26.
 - Cumulative; the final emphasizes post-MT2 material (Scheme, interpreters, SQL) but reuses everything.
- **Core idea (one sentence):** Strip programming down to a small set of evaluation rules (the environment model), then build up the same idea in three different paradigms — functional, object-oriented, and declarative — so you can read, write, and *interpret* code in any of them.
- **What makes the final different from MT1/MT2:** MT1 was tracing + functional. MT2 was OOP + linked structures + start of Scheme. The **final adds Scheme depth (incl. interpreters, tail calls, streams) + SQL**, and it expects you to have all of MT1/MT2 still cold.

2. Topics — every lecture this semester

Slide deck filenames correspond to lecture numbers. Some numbers are missing in your folder (08, 13, 24–25, 27, 36) — these are typically holidays, exam days, or guest lectures that don't have downloadable slides. The full course schedule is on cs61a.org; the missing topics that matter are noted below.

Pre-MT1 material (Slides 01–07, 09–10) — Functional & Recursion

#	Slide	Topic / main idea
01	Welcome	Course logistics. Skip.
02	Functions	Function definitions, expressions vs statements, pure vs non-pure functions, return vs print, signatures.
03	Control	Conditionals, boolean operators, short-circuit evaluation, control flow.
04	Higher-Order Functions	Functions as first-class values, returning functions, lambda, closure intuition.
05	Environments	The environment model: frames, parents, name lookup, evaluation order. Foundation for tracing.
06	Functional Abstraction	Abstraction barriers, function composition, decomposition principle.
07	Function Examples	Worked examples consolidating HOFs and recursion.
09	Recursion	Recursive function design, base case + recursive case, recursion as proof by induction.
10	Tree	Multiple recursive calls per call, Fibonacci, counting partitions,

#	Slide	Topic / main idea
	Recursion	exponential blowup.

MT1 → MT2 material (Slides 11–23) — Sequences, Mutability, OOP

#	Slide	Topic / main idea
11	Sequences	Lists, ranges, slicing, list comprehensions, sum/max/etc as HOFs over sequences.
12	Containers	Tuples, dicts, sets — when to use which, immutability of tuples vs lists.
14	Trees	Tree data abstraction, recursive tree functions, Tree class definition.
15	Mutability	Mutating lists, aliasing, identity (is) vs equality (==), reference semantics.
16	Iterators	iter, next, __iter__, StopIteration, what makes something iterable.
17	Generators	yield, generator functions, lazy evaluation, generator expressions.
18	Objects	OOP basics, classes vs instances, methods, self.
19	Attributes	Attribute lookup, instance vs class attributes, attribute assignment semantics.
20	Inheritance	Subclasses, super(), method resolution order, isinstance.
21	Representation	__repr__, __str__, polymorphism, special methods.
22	Composition	Composing objects, Linked Lists as a class, Tree as a class.
23	Decomposition	Designing class hierarchies, separation of responsibility.

Post-MT2 material (Slides 26, 28–35, 37) — Scheme, Interpreters, SQL — HIGHEST FINAL EMPHASIS

#	Slide	Topic / main idea
26	Scheme	Scheme syntax, special forms (define, if, cond, let, lambda), pairs and lists, cons/car/cdr.
(24–25 missing in folder)	(Likely Tail Calls + Streams)	Probably covered on cs61a.org schedule — verify. Tail calls = constant-space recursion (tail-call optimized only in Scheme, not Python). Streams = lazy infinite lists in Scheme. Both are common final topics.
28	Calculator	The Calculator interpreter — read/parse/eval cycle, simple arithmetic-only language.
29	Interpreters	Full Scheme interpreter — scheme_eval, scheme_apply, environments, special forms vs procedures.
30	Programs as Data	Code-is-data, manipulating Scheme programs as Pairs, quote/unquote intuition.
31	Macros	Macros = code that writes code at parse time, define-macro, hygiene caveats.
32	SQL	SQL basics: SELECT, FROM, WHERE, simple queries on a single table.

#	Slide	Topic / main idea
33	Tables	Multi-table reasoning, joins (FROM A, B), self-joins for parent/child relationships.
34	Aggregation	GROUP BY, HAVING, aggregate functions (COUNT, MAX, SUM, AVG).
35	Databases	CREATE TABLE, INSERT, recursive CTEs (WITH RECURSIVE), database design intuition.
37	Designing Functions	Wrap-up: how to design well-decomposed Python functions. Lightweight; mostly review.

What's missing from your folder that you'd find on cs61a.org

- Lecture 08, 13: usually exam-day or holiday
- Lectures 24–25: likely **Tail Calls** and **Streams** (or similar Scheme topics) — high probability final topic
- Lecture 27: usually **Exceptions** or pre-Calculator review
- Lecture 36: usually **Final Review**

Action: On your first review session for CS 61A (Day 1 evening, May 4), open cs61a.org and confirm what these missing slide numbers actually cover. Don't skip — Tail Calls and Streams are nearly always tested on the final.

3. Priority list

Three tiers. Within tiers, ranked by exam-likelihood × payoff per study hour. CS 61A finals reliably have the topic-weight roughly: WWPD/tracing 25%, Python implementation 25%, Scheme 25%, SQL 15%, conceptual 10%. The priorities below reflect that.

TIER 1 — HIGH (~55% of CS 61A time)

1. **Scheme (Slides 26 + missing 24–25 + 28–31).** Largest single block on the final by tradition. Cover: `lambda`, `define`, `cond`, `let`, tail calls (constant-space recursion), streams (lazy infinite lists), interpreters, programs-as-data. *Exam likelihood: ~99%.*
2. **Environment Diagrams (Slides 04–05, 18–22).** Always tested, mechanical, full points if practiced. Includes nested `def`, `lambda`, OOP attribute lookup. *Exam likelihood: ~99%.*
3. **Recursion + Tree Recursion + Linked Lists / Trees (Slides 09, 10, 14, 22).** Implementation problems writing recursive functions on Python `Link/Tree` classes (or their Scheme equivalents). *Exam likelihood: ~95%.*
4. **SQL (Slides 32–35).** Mechanical points: write a query, predict output of a query, GROUP BY + HAVING patterns, recursive CTEs. *Exam likelihood: ~95%.*
5. **OOP — inheritance, attribute lookup, mutation (Slides 18–22).** Often combined with Scheme/Python implementation problems. Marcus's MT2_Code_Trigger_Sheet.md already covers this well. *Exam likelihood: ~85%.*

TIER 2 — MEDIUM (~30% of time)

6. **Higher-Order Functions + lambda (Slides 04, 06).** Almost always 1 problem on the final, often as a "rewrite this with HOFs" or closure-puzzle. *Exam likelihood: ~80%.*
7. **Iterators + Generators (Slides 16–17).** Common medium-difficulty problem: implement a generator that produces something. *Exam likelihood: ~70%.*
8. **Mutability + identity (Slide 15).** Usually 1 short trace question. Marcus's MT1 reference sheet covers this. *Exam likelihood: ~60%.*

9. **Calculator interpreter (Slide 28).** Smaller version of the full Scheme interpreter — final often has 1–2 questions modifying it. *Exam likelihood: ~65%.*
10. **Macros (Slide 31).** Smaller in scope but tested; "predict what this macro expands to." *Exam likelihood: ~55%.*

TIER 3 — LOW (~10% of time, mostly skim)

11. **Sequences and Containers (Slides 11–12).** Cumulative baseline; rarely a question on its own — folded into Tier 1 problems. *Exam likelihood: ~30% as standalone.*
12. **Control flow + Functions basics (Slides 02–03).** Pure foundation. Skim the MT1 reference sheet, don't open the slides. *Exam likelihood: ~20% standalone.*
13. **Designing Functions (Slide 37).** Wrap-up; conceptual only. *Exam likelihood: ~15%.*
14. **Welcome (Slide 01).** Skip. *Exam likelihood: 0%.*

Critical leverage you already have

- Marcus's `61A_Exam_Reference.md` covers MT1/MT2 patterns at the "see X → write Y" level. **This is your primary study artifact, not the slides.** Re-read it cold; if any pattern doesn't trigger instantly, drill that pattern.
- `MT2_Code_Trigger_Sheet.md` for Linked Lists/Trees/OOP — same approach: re-read, drill anything that doesn't trigger.
- 16 past exams. Pick `61a-fa25-final.pdf` and `61a-sp25-final_sol.pdf` as the highest-signal practice exams.

4. Summary, workload anticipation, and time-allocation advice

Concise summary

You're well-prepared for everything pre-MT2 (your own MD reference sheets are excellent and your project codebase shows you've done the work). The final's leverage points are the **post-MT2 material** — Scheme, Tail Calls, Streams, Interpreters, SQL — plus mechanical tracing of environment diagrams. CS 61A's final is the most predictable of the three CS-style courses; question patterns repeat semester-to-semester.

Anticipated review workload (honest)

- Total hours for comfortable A-level: **10–14 hours.**
- Master plan allocates 10 hr in 5 days + 2 hr May 9 buffer + ~2 hr May 11 evening ≈ 14 hr. **This is right at the upper end of what's needed** — given you've completed the Scheme project and have study guides, you have slack here.

Hour-by-hour suggested split within those ~14 hours

Activity	Hours	Why
Re-read your own <code>61A_Exam_Reference.md</code> cold	0.5	Free recall test — flag patterns that don't trigger.
Re-read your <code>MT2_Code_Trigger_Sheet.md</code> cold	0.5	Same, for OOP/Linked/Tree patterns.
Re-read Slide 26 (Scheme) + write 5 Scheme functions on paper (factorial, length, accumulate, count, tree-leaves)	1.5	Mechanical fluency for Scheme syntax.
Re-read Slides 28–30 (Interpreters + Programs as Data); trace eval/apply on a small expression	1.5	Interpreters always on final.
Tail calls: write 3 iterative→tail-recursive Scheme conversions	1	High-yield if Slide 24 is indeed tail calls.

Activity	Hours	Why
Streams: implement stream-map, stream-filter, stream-take in Scheme	1	Same.
SQL drills: 5 queries (joins, GROUP BY + HAVING, recursive CTE) from past finals	1.5	Mechanical points.
Environment diagrams: 5 cold from Marcus's MT1 reference patterns	1	Free points if drilled.
Take 61a-fa25-final.pdf timed (~3 hr) — review every wrong answer	4	Single highest-yield activity.
Cheat sheet — handwritten 2 pages	1.5	Writing IS review. Focus on: Scheme syntax cheats, SQL keyword patterns, Linked/Tree class definitions.

Total $\approx$ 14 hr.

Time-allocation advice — adjustments to the master plan

15. **You can safely take 1–2 hours from CS 61A and give them to ECON 100B or ECON 131** if those need it. Your existing MD references mean CS 61A is the slack course of the four.
16. **Day 3 (Wed May 6) morning is currently a 4 hr CS 61A block** — that's good for Interpreters + Calculator deep-dive. Use the first 2 hours to read Slides 28–30 carefully + trace Calculator code, then the remaining 2 hours doing 2 past-final Scheme problems.
17. **Take a past final timed BEFORE Day 5 cheat-sheet writing.** A past-final attempt reveals what to put on the cheat sheet. The master plan has the timed past final on Day 4 for ECON 100B — schedule a CS 61A timed past final on Day 4 evening (the 1 hr CS 61B "evening" block can be replaced; CS 61B has more practice exams and you're doing a CS 61B past final on a different day).
18. **Don't skip Tail Calls and Streams just because slide 24/25 are missing from your folder.** Open cs61a.org textbook (Composing Programs) chapters 4 and 5 — they cover this exactly. ~30 min of textbook reading replaces the missing slides.

What NOT to do (anti-advice)

- Don't re-read Slides 02–03, 11–12 from scratch. They're cumulative baseline; you've used these patterns in projects.
- Don't try to write the cheat sheet without first taking a past final. The act of taking the exam tells you what belongs on the sheet.
- Don't memorize the full Scheme `eval/apply` source code — understand the *flow* (special forms vs procedure calls) so you can modify it, not regurgitate it.
- Don't open the AntsProject — it's done, it's the oral-exam version, it's not on the final.